imports

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import re
from collections import Counter
from wordcloud import WordCloud
import geopandas as gpd
import warnings
warnings.filterwarnings('ignore')
```

Part A - Cleaning & Sentiment Overview

Data Cleaning

The dataset contains 2000 climate-related tweets. Before visualizing, we need to:

- Parse dates properly
- Detect and handle duplicates
- Handle missing values in like_count and retweet_count

A1 - Load and Inspect

In [2]:
```
# load the dataset
df = pd.read_csv('Climate_Tweets_Sample.csv')

# initial inspection
print(f"Shape: {df.shape}")
df.head()
```

```
Shape: (2000, 8)
```

Out[2]:

	tweet_id	date	country	sentiment	keyword	text	retwe
0	2803826677020224	2023-11-01	Denmark	negative	air quality	Breaking: concerns grow over air quality; data...	
1	1119349365897942	2022-11-16	Spain	negative	air quality	Reminder: concerns grow over air quality; deta...	
2	3605591970800719	2023-01-20	France	neutral	green transition	FYI: discussion continues on green transition;...	
3	4546715437012377	2023-09-10	Saudi Arabia	negative	green transition	Opinion: concerns grow over green transition; ...	
4	9630049315224554	2022-09-26	Ireland	negative	global warming	Opinion: global warming impacts are worsening;...	

In [3]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   tweet_id       2000 non-null   int64
 1   date           2000 non-null   str
 2   country        2000 non-null   str
 3   sentiment      2000 non-null   str
 4   keyword        2000 non-null   str
 5   text           2000 non-null   str
 6   retweet_count  1962 non-null   float64
 7   like_count     1968 non-null   float64
dtypes: float64(2), int64(1), str(5)
memory usage: 125.1 KB
```

In [4]:
```
df.describe(include='all')
```

Out[4]:

	tweet_id	date	country	sentiment	keyword	text	retwe
count	2.000000e+03	2000	2000	2000	2000	2000	196
unique	NaN	915	39	3	14	1840	
top	NaN	2022-05-26	New Zealand	negative	emissions	Report: encouraging signs for net zero; data a...	
freq	NaN	8	69	736	158	4	
mean	5.539795e+15	NaN	NaN	NaN	NaN	NaN	4
std	2.605952e+15	NaN	NaN	NaN	NaN	NaN	2
min	1.002787e+15	NaN	NaN	NaN	NaN	NaN	
25%	3.343202e+15	NaN	NaN	NaN	NaN	NaN	2
50%	5.547012e+15	NaN	NaN	NaN	NaN	NaN	4
75%	7.797005e+15	NaN	NaN	NaN	NaN	NaN	6
max	9.993858e+15	NaN	NaN	NaN	NaN	NaN	33

A2 - Date Parsing

Convert the date column from string to datetime for proper time-series operations.

In [5]:

```
# check current date format
print(f"Current dtype: {df['date'].dtype}")
print(f"Sample values: {df['date'].head(5).tolist()}")

# convert to datetime
df['date'] = pd.to_datetime(df['date'], format='%Y-%m-%d')

# verify conversion
print(f"\nNew dtype: {df['date'].dtype}")
print(f"Date range: {df['date'].min()} to {df['date'].max()}")
```

```
Current dtype: str
Sample values: ['2023-11-01', '2022-11-16', '2023-01-20', '2023-09-10', '202
2-09-26']

New dtype: datetime64[us]
Date range: 2021-01-03 00:00:00 to 2023-12-31 00:00:00
```

A3 - Duplicate Detection and Handling

Check for both full-row duplicates and text-level duplicates.

In [6]:

```
# check for exact row duplicates
print(f"Exact row duplicates: {df.duplicated().sum()}")
```

```
print(f"Duplicate tweet_ids: {df.duplicated(subset=['tweet_id']).sum()}")

# check for duplicate tweet text
text_dupes = df[df.duplicated(subset=['text'], keep=False)].sort_values('tex
print(f"\nRows with duplicate text: {len(text_dupes)}")
print(f"Unique duplicate texts: {text_dupes['text'].nunique()}")

# show example
print("\nExample duplicate text group:")
sample_text = text_dupes['text'].value_counts().index[0]
print(df[df['text'] == sample_text][['tweet_id', 'date', 'country', 'sentime
```

```
Exact row duplicates: 0
Duplicate tweet_ids: 0

Rows with duplicate text: 305
Unique duplicate texts: 145

Example duplicate text group:
              tweet_id       date        country sentiment  \
43    3593653781002441 2021-06-25       Colombia  positive
83    5313876958018246 2023-08-22    Philippines  positive
504   7572448285169845 2021-11-21          Japan  positive
1648  2854259649959415 2022-05-29  United States  positive

                                                  text
43    Report: encouraging signs for net zero; data a...
83    Report: encouraging signs for net zero; data a...
504   Report: encouraging signs for net zero; data a...
1648  Report: encouraging signs for net zero; data a...
```

In [7]:

```
# drop text-level duplicates, keeping the first occurrence
before = len(df)
df = df.drop_duplicates(subset=['text'], keep='first')
after = len(df)
print(f"Rows before: {before}")
print(f"Rows after: {after}")
print(f"Removed: {before - after} duplicate text entries")
```

```
Rows before: 2000
Rows after: 1840
Removed: 160 duplicate text entries
```

Justification: 160 rows share identical tweet text with different tweet_ids, countries, and dates. These are likely reposted or templated tweets. Since the text content is identical, keeping all copies would inflate word frequency counts and sentiment distributions. I keep the first occurrence of each unique text and drop the rest.

A4 - Handle Missing Values

Check and handle missing values in like_count and retweet_count.

In [8]:
```
# check missing values after duplicate removal
print("Missing values per column:")
print(df.isnull().sum())
print(f"\nTotal rows: {len(df)}")
print(f"Missing retweet_count: {df['retweet_count'].isna().sum()} ({df['retw
print(f"Missing like_count: {df['like_count'].isna().sum()} ({df['like_count
```

```
Missing values per column:
tweet_id          0
date              0
country           0
sentiment         0
keyword           0
text              0
retweet_count    33
like_count       30
dtype: int64

Total rows: 1840
Missing retweet_count: 33 (1.8%)
Missing like_count: 30 (1.6%)
```

In [9]:
```
# impute missing values with median
df['retweet_count'] = df['retweet_count'].fillna(df['retweet_count'].median(
df['like_count'] = df['like_count'].fillna(df['like_count'].median())

# cast to int since counts are whole numbers
df['retweet_count'] = df['retweet_count'].astype(int)
df['like_count'] = df['like_count'].astype(int)

# verify
print("Missing values after imputation:")
print(df.isnull().sum())
print(f"\nData types:")
print(df.dtypes)
```

```
Missing values after imputation:
tweet_id         0
date             0
country          0
sentiment        0
keyword          0
text             0
retweet_count    0
like_count       0
dtype: int64

Data types:
tweet_id                  int64
date             datetime64[us]
country                     str
sentiment                   str
keyword                     str
text                        str
retweet_count             int64
like_count                int64
dtype: object
```

Justification: Missing values in retweet_count (33 rows, ~1.8%) and like_count (30 rows, ~1.6%) represent a small fraction of the dataset. I chose median imputation because it preserves all rows for downstream analysis (sentiment counts, word clouds, etc.), the median is robust to the right-skewed distribution of engagement metrics (retweet max = 330, like max = 867), and the missing proportion is small enough that imputation won't meaningfully distort the data.

Cleaning Summary

In [10]:
```
print("=== CLEANING SUMMARY ===")
print(f"Final shape: {df.shape}")
print(f"Date range: {df['date'].min().date()} to {df['date'].max().date()}")
print(f"Missing values: {df.isnull().sum().sum()}")
print(f"Sentiments: {df['sentiment'].value_counts().to_dict()}")
print(f"Countries: {df['country'].nunique()}")
print(f"Keywords: {df['keyword'].nunique()}")

# save cleaned dataset
df.to_csv('Climate_Tweets_Cleaned.csv', index=False)
print("\nCleaned dataset saved.")
```

```
=== CLEANING SUMMARY ===
Final shape: (1840, 8)
Date range: 2021-01-03 to 2023-12-31
Missing values: 0
Sentiments: {'negative': 668, 'positive': 608, 'neutral': 564}
Countries: 39
Keywords: 14

Cleaned dataset saved.
```

Sentiment Visualization

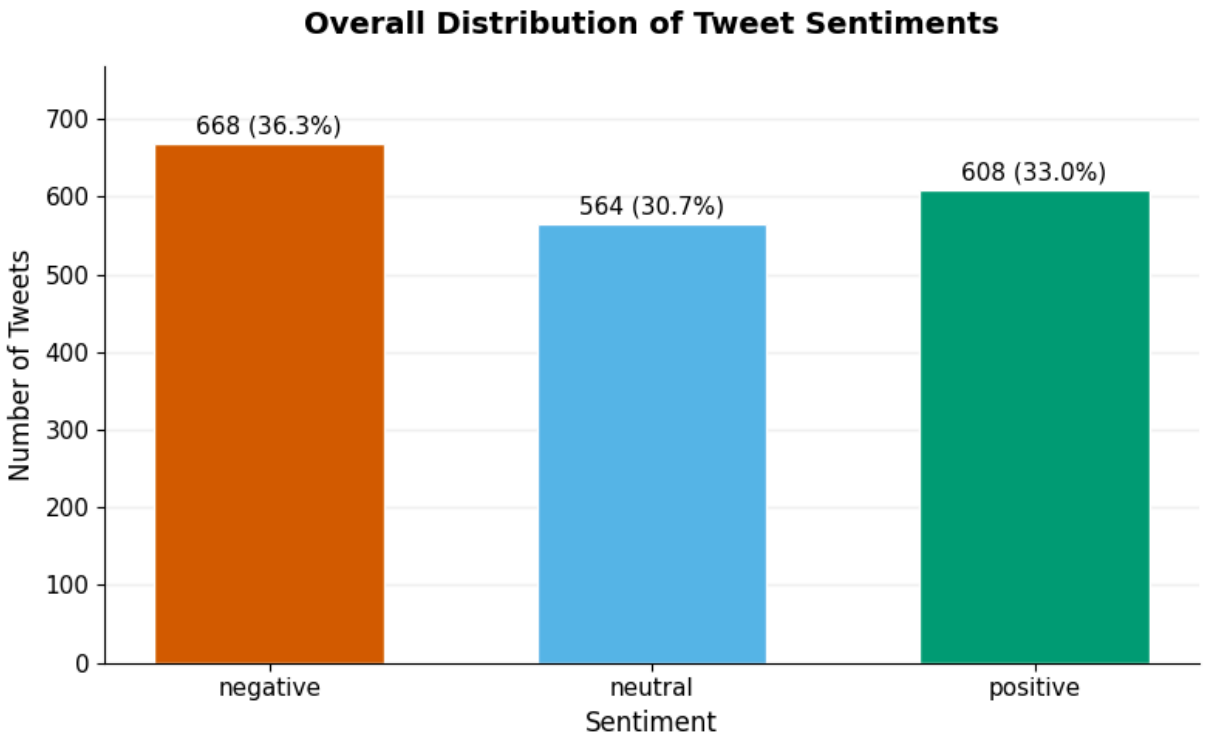
A5 - Overall Sentiment Distribution

Bar chart showing the distribution of tweet sentiments across all countries.

In [11]:
```
# count sentiments
sentiment_counts = df['sentiment'].value_counts().sort_index()
print(sentiment_counts)
```

```
sentiment
negative    668
neutral     564
positive    608
Name: count, dtype: int64
```

In [12]:
```
# bar chart: overall sentiment distribution
# Okabe-Ito palette, colourblind friendly
sentiment_colors = {
    'negative': '#D55E00',
    'neutral': '#56B4E9',
    'positive': '#009E73'
}

fig, ax = plt.subplots(figsize=(8, 5))

bars = ax.bar(sentiment_counts.index,
              sentiment_counts.values,
              color=[sentiment_colors[s] for s in sentiment_counts.index],
              edgecolor='white',
              width=0.6)

# value labels on top of bars
for bar in bars:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, height + 8,
            f'{int(height)} ({height/len(df)*100:.1f}%)',
            ha='center', va='bottom', fontsize=11)

ax.set_xlabel('Sentiment', fontsize=12)
ax.set_ylabel('Number of Tweets', fontsize=12)
ax.set_title('Overall Distribution of Tweet Sentiments', fontsize=14, fontwe
ax.tick_params(axis='both', labelsize=11)
ax.set_ylim(0, sentiment_counts.max() * 1.15)

for spine in ['top', 'right']:
    ax.spines[spine].set_visible(False)

ax.yaxis.grid(True, linestyle='-', alpha=0.15)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()
```



A6 - Sentiment Distribution by Top 10 Countries

Identify the top 10 countries with the most tweets, then visualize sentiment breakdown.

In [13]:
```
# identify top 10 countries by tweet count
top10_countries = df['country'].value_counts().head(10)
print("Top 10 countries by tweet count:")
print(top10_countries)
```

```
Top 10 countries by tweet count:
country
New Zealand     65
Thailand        65
Mexico          62
Spain           60
Denmark         57
Saudi Arabia    54
Chile           54
Peru            52
China           51
Italy           51
Name: count, dtype: int64
```

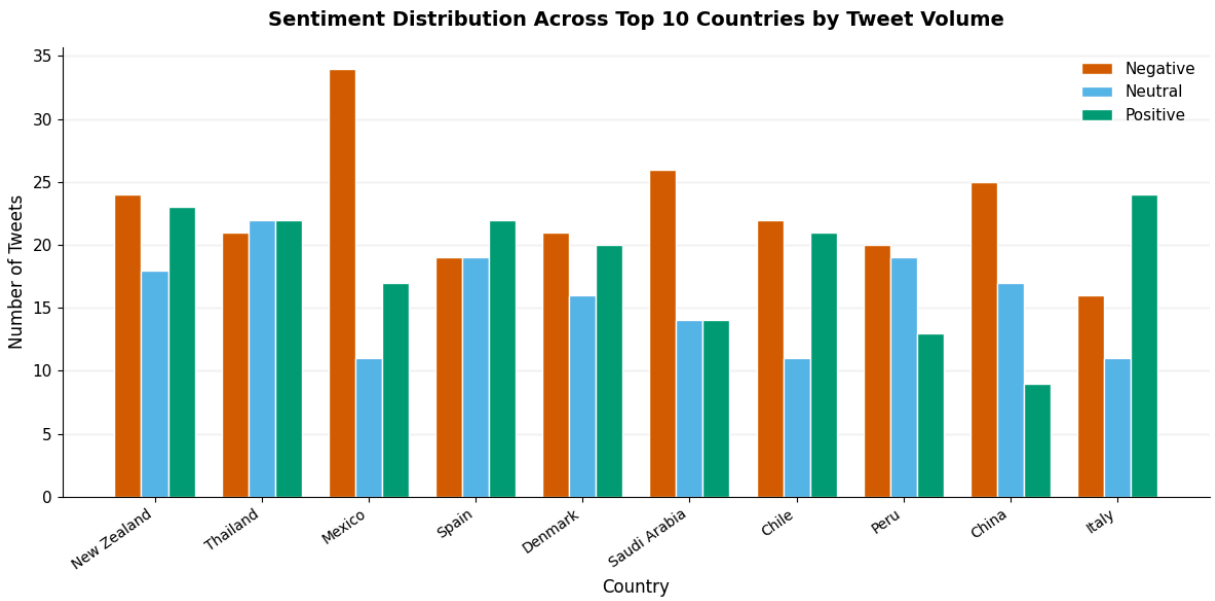
In [14]:
```
# filter to top 10 countries and build sentiment crosstab
df_top10 = df[df['country'].isin(top10_countries.index)]
sentiment_by_country = pd.crosstab(df_top10['country'], df_top10['sentiment'

# sort by total tweets descending
sentiment_by_country = sentiment_by_country.loc[top10_countries.index]
```

```
print(sentiment_by_country)
```

```
sentiment     negative  neutral  positive
country
New Zealand         24       18        23
Thailand            21       22        22
Mexico              34       11        17
Spain               19       19        22
Denmark             21       16        20
Saudi Arabia        26       14        14
Chile               22       11        21
Peru                20       19        13
China               25       17         9
Italy               16       11        24
```

In [15]:

```
# grouped bar chart: sentiment distribution across top 10 countries
# Okabe-Ito palette, colourblind friendly
fig, ax = plt.subplots(figsize=(12, 6))

x = np.arange(len(sentiment_by_country))
width = 0.25

for i, (sentiment, color) in enumerate(sentiment_colors.items()):
    if sentiment in sentiment_by_country.columns:
        values = sentiment_by_country[sentiment].values
        bars = ax.bar(x + i * width, values, width,
                      label=sentiment.capitalize(), color=color, edgecolor='

ax.set_xlabel('Country', fontsize=12)
ax.set_ylabel('Number of Tweets', fontsize=12)
ax.set_title('Sentiment Distribution Across Top 10 Countries by Tweet Volume
             fontsize=14, fontweight='bold', pad=15)
ax.set_xticks(x + width)
ax.set_xticklabels(sentiment_by_country.index, rotation=35, ha='right', font
ax.legend(frameon=False, fontsize=11)
ax.tick_params(axis='y', labelsize=11)

for spine in ['top', 'right']:
    ax.spines[spine].set_visible(False)

ax.yaxis.grid(True, linestyle='-', alpha=0.15)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()
```



Part B - Text Preprocessing & Word Cloud

Text Preprocessing

Steps:

1. Lowercase all text
2. Remove punctuation and numbers
3. Remove stopwords (English common words + domain-specific template filler)
4. Filter tokens shorter than 3 characters

In [16]:
```
# define stopwords manually (NLTK unavailable in this environment)
STOPWORDS = set("""i me my myself we our ours ourselves you your yours yours
he him his himself she her hers herself it its itself they them their theirs
what which who whom this that these those am is are was were be been being h
having do does did doing a an the and but if or because as until while of at
about against between through during before after above below to from up dow
off over under again further then once here there when where why how all bot
more most other some such no nor not only own same so than too very s t can
should now d ll m o re ve y ain aren couldn didn doesn hadn hasn haven isn m
needn shan shouldn wasn weren won wouldn could would also much get""".split(

# domain-specific stopwords: template filler words that appear across all se
# without carrying meaningful climate content
STOPWORDS.update(['check', 'figures', 'details', 'incoming', 'source', 'comm
                  'data', 'attached', 'learn', 'soon', 'share', 'responsibly
                  'stay', 'informed', 'think', 'report', 'update', 'opinion'
                  'breaking', 'reminder', 'heads', 'fyi', 'research', 'new',
                  'overview', 'trends', 'latest', 'face', 'discussion', 'cor
                  'released', 'stats'])
```

```python
def preprocess_text(text):
    text = text.lower()                          # lowercase
    text = re.sub(r'[^\w\s]', '', text)          # remove punctuation
    text = re.sub(r'\d+', '', text)               # remove numbers
    tokens = text.split()
    tokens = [t for t in tokens if t not in STOPWORDS and len(t) > 2]
    return tokens

# apply preprocessing
df['tokens'] = df['text'].apply(preprocess_text)

# check results
all_tokens = [t for tokens in df['tokens'] for t in tokens]
print(f"Total tokens after preprocessing: {len(all_tokens)}")
print(f"Unique tokens: {len(set(all_tokens))}")
print(f"\nSample preprocessed tweet:")
print(f"  Original: {df['text'].iloc[0]}")
print(f"  Tokens:   {df['tokens'].iloc[0]}")
```

```
Total tokens after preprocessing: 5744
Unique tokens: 39

Sample preprocessed tweet:
  Original: Breaking: concerns grow over air quality; data attached.
  Tokens:   ['concerns', 'grow', 'air', 'quality']
```

Preprocessing choices:

- **Lowercasing** standardizes case for frequency counting.
- **Punctuation removal** strips noise characters (colons, semicolons, periods) common in the tweet format.
- **Stopwords** combine a standard English stopword list with domain-specific filler words ("Breaking:", "Report:", "FYI:", "details incoming", "overview", "trends", etc.) that appear in the templated tweet structure and don't carry topical meaning.
- **Minimum token length of 3** removes residual two-letter fragments.
- I opted not to lemmatize because the key terms in this dataset are mostly base-form nouns and adjectives (emissions, drought, warming) that don't benefit from stemming.

Keyword & Word Cloud Analysis

B1 - Top 20 Most Frequent Terms

Identify and visualize the 20 most common words across all tweets.

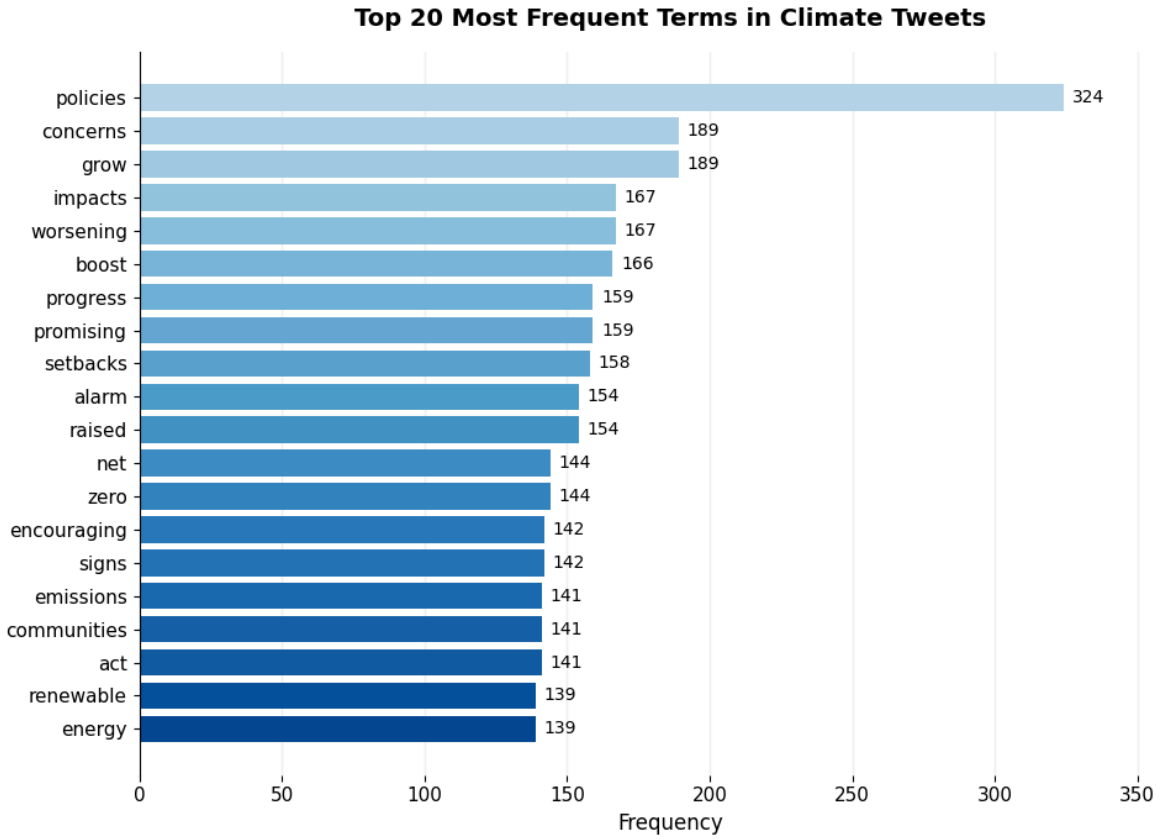
In [17]:
```python
# top 20 most frequent terms
top20 = Counter(all_tokens).most_common(20)
terms, counts = zip(*top20)

print("Top 20 terms:")
```

```
for term, count in top20:
    print(f"  {term:20s} {count}")
```

```
Top 20 terms:
  policies             324
  concerns             189
  grow                 189
  impacts              167
  worsening            167
  boost                166
  progress             159
  promising            159
  setbacks             158
  alarm                154
  raised               154
  net                  144
  zero                 144
  encouraging          142
  signs                142
  emissions            141
  communities          141
  act                  141
  renewable            139
  energy               139
```

In [18]:

```
# bar chart: top 20 most frequent terms
fig, ax = plt.subplots(figsize=(10, 7))

colors = plt.cm.Blues(np.linspace(0.3, 0.9, len(terms)))[::-1]
bars = ax.barh(list(reversed(terms)), list(reversed(counts)), color=colors)

# value labels
for bar in bars:
    width = bar.get_width()
    ax.text(width + 3, bar.get_y() + bar.get_height()/2,
            f'{int(width)}', ha='left', va='center', fontsize=10)

ax.set_xlabel('Frequency', fontsize=12)
ax.set_title('Top 20 Most Frequent Terms in Climate Tweets', fontsize=14, fo
ax.tick_params(axis='both', labelsize=11)

for spine in ['top', 'right', 'bottom']:
    ax.spines[spine].set_visible(False)

ax.xaxis.grid(True, linestyle='-', alpha=0.15)
ax.set_axisbelow(True)
ax.set_xlim(0, max(counts) * 1.15)

plt.tight_layout()
plt.show()
```



B2 - Word Clouds by Sentiment

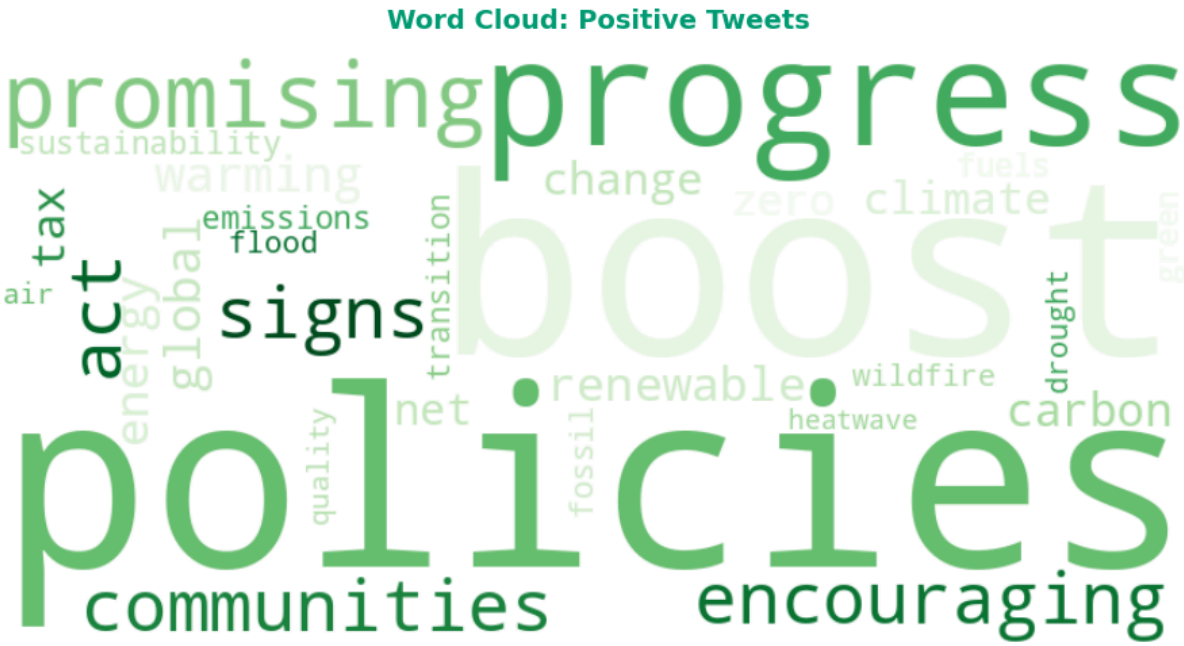
Separate word clouds for positive and negative tweets to compare language patterns.

In [19]:
```
# separate tokens by sentiment
pos_tokens = [t for tokens, s in zip(df['tokens'], df['sentiment']) if s ==
neg_tokens = [t for tokens, s in zip(df['tokens'], df['sentiment']) if s ==

print(f"Positive tokens: {len(pos_tokens)}")
print(f"Negative tokens: {len(neg_tokens)}")
print(f"\nTop 10 positive: {Counter(pos_tokens).most_common(10)}")
print(f"Top 10 negative: {Counter(neg_tokens).most_common(10)}")
```

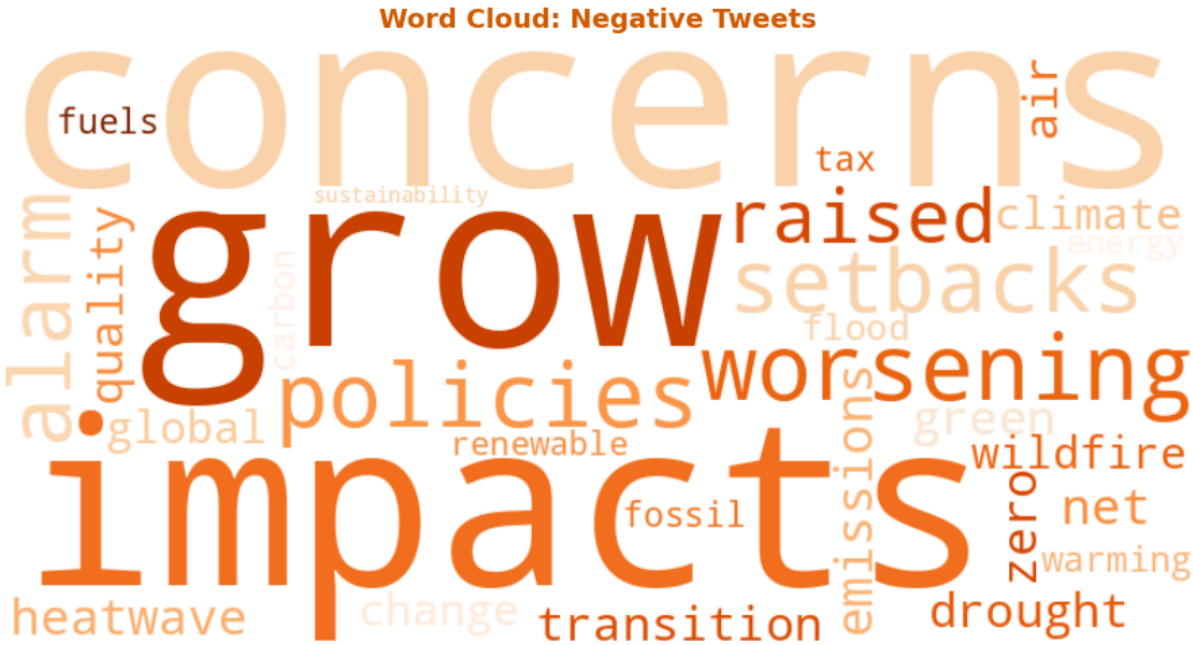
```
Positive tokens: 2194
Negative tokens: 2377

Top 10 positive: [('policies', 166), ('boost', 166), ('progress', 159), ('pr
omising', 159), ('encouraging', 142), ('signs', 142), ('communities', 141),
('act', 141), ('global', 55), ('warming', 55)]
Top 10 negative: [('concerns', 189), ('grow', 189), ('impacts', 167), ('wors
ening', 167), ('policies', 158), ('setbacks', 158), ('alarm', 154), ('raise
d', 154), ('emissions', 57), ('heatwave', 56)]
```

In [20]:
```
# word cloud: positive tweets
wc_pos = WordCloud(width=800, height=400, background_color='white',
                   colormap='Greens', max_words=80,
                   collocations=False,
                   prefer_horizontal=0.7).generate(' '.join(pos_tokens))
```

```
fig, ax = plt.subplots(figsize=(10, 5))
ax.imshow(wc_pos, interpolation='bilinear')
ax.set_title('Word Cloud: Positive Tweets', fontsize=14, fontweight='bold',
ax.axis('off')

plt.tight_layout()
plt.show()
```



In [21]:
```
# word cloud: negative tweets
wc_neg = WordCloud(width=800, height=400, background_color='white',
                   colormap='Oranges', max_words=80,
                   collocations=False,
                   prefer_horizontal=0.7).generate(' '.join(neg_tokens))

fig, ax = plt.subplots(figsize=(10, 5))
ax.imshow(wc_neg, interpolation='bilinear')
ax.set_title('Word Cloud: Negative Tweets', fontsize=14, fontweight='bold',
ax.axis('off')

plt.tight_layout()
plt.show()
```



Observations: The positive word cloud is dominated by terms like "encouraging", "signs", "progress", "promising", and "boost", reflecting optimistic framing around policies and sustainability. The negative cloud emphasizes "concerns", "worsening", "impacts", "setbacks", and "alarm", consistent with pessimistic coverage of environmental decline. Both clouds share core topic words (emissions, climate, energy) but differ sharply in their framing verbs and adjectives.

Part C - Geospatial Visualization

Choropleth map showing the number of climate tweets per country.

Country Name Standardization

In [22]:
```
# count tweets per country
country_counts = df['country'].value_counts().reset_index()
country_counts.columns = ['country', 'tweet_count']

# country name mapping for Natural Earth shapefile
name_map = {
    'United States': 'United States of America',
}

country_counts['map_name'] = country_counts['country'].map(lambda x: name_ma

# load world shapefile (built into geopandas)
world = gpd.read_file(gpd.datasets.get_path('naturalearth_lowres'))

# check for unmatched countries
unmatched = [c for c in country_counts['map_name'] if c not in world['name']
```

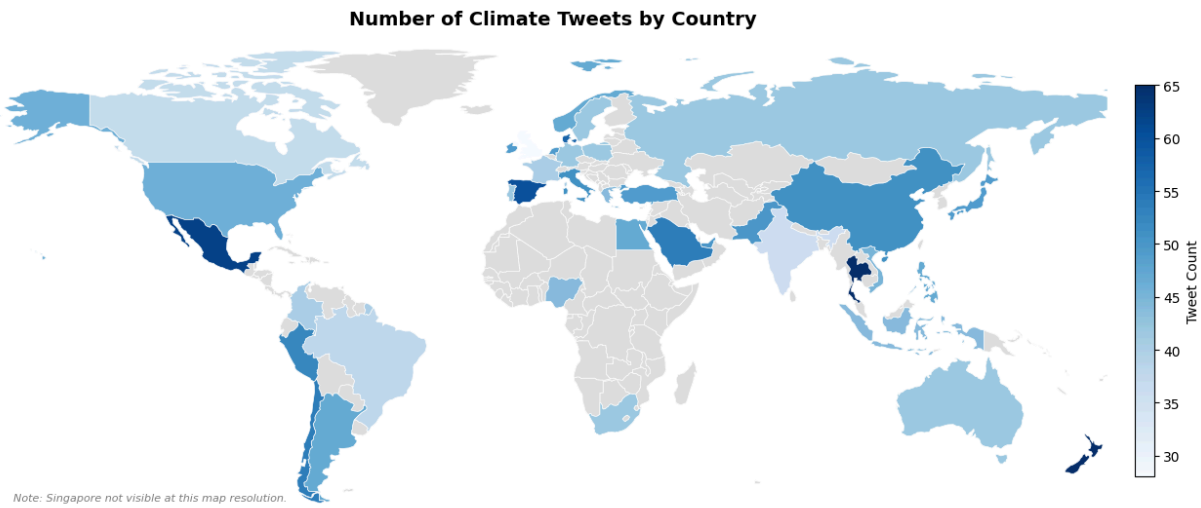
```
matched = len(country_counts) - len(unmatched)
print(f"Matched: {matched} / {len(country_counts)}")
print(f"Unmatched: {unmatched}")

# merge
merged = world.merge(country_counts, left_on='name', right_on='map_name', ho
```

```
Matched: 38 / 39
Unmatched: ['Singapore']
```

Country name handling: The only inconsistency is "United States" vs "United States of America" in the shapefile. Singapore is a city-state too small to render at this map resolution and doesn't appear in the Natural Earth low-res dataset, so it's excluded (1 country out of 39).

Choropleth Map

In [23]:
```
# choropleth: tweet count by country
fig, ax = plt.subplots(figsize=(14, 7))

# countries without data in light gray
merged[merged['tweet_count'].isna()].plot(ax=ax, color='#E0E0E0', edgecolor=

# countries with data
merged_data = merged[merged['tweet_count'].notna()]
merged_data.plot(column='tweet_count', ax=ax, legend=True,
                 cmap='Blues', edgecolor='white', linewidth=0.5,
                 legend_kwds={'label': 'Tweet Count',
                              'orientation': 'vertical',
                              'shrink': 0.6,
                              'pad': 0.02})

ax.set_title('Number of Climate Tweets by Country', fontsize=14, fontweight=
ax.set_xlim(-170, 180)
ax.set_ylim(-60, 85)
ax.axis('off')

# note about Singapore
ax.text(-165, -55, 'Note: Singapore not visible at this map resolution.',
        fontsize=8, color='gray', style='italic')

plt.tight_layout()
plt.show()
```



Part D - Uncertainty & Association

D1 - Monthly Negative Sentiment Rate + 95% Confidence Interval

Compute the monthly proportion of negative tweets and visualize with Wilson score confidence intervals.

In [24]:
```
# compute monthly negative sentiment rate
df['month'] = df['date'].dt.to_period('M')
df['is_negative'] = (df['sentiment'] == 'negative').astype(int)

monthly = df.groupby('month').agg(
    n_negative=('is_negative', 'sum'),
    n_total=('is_negative', 'count')
).reset_index()

monthly['rate'] = monthly['n_negative'] / monthly['n_total']

# Wilson score confidence interval
z = 1.96  # 95% CI
n = monthly['n_total']
p = monthly['rate']

monthly['ci_low'] = (p + z**2/(2*n) - z * np.sqrt((p*(1-p) + z**2/(4*n))/n))
monthly['ci_high'] = (p + z**2/(2*n) + z * np.sqrt((p*(1-p) + z**2/(4*n))/n)

monthly['month_dt'] = monthly['month'].dt.to_timestamp()

# check results
print(monthly[['month', 'n_negative', 'n_total', 'rate', 'ci_low', 'ci_high'
```

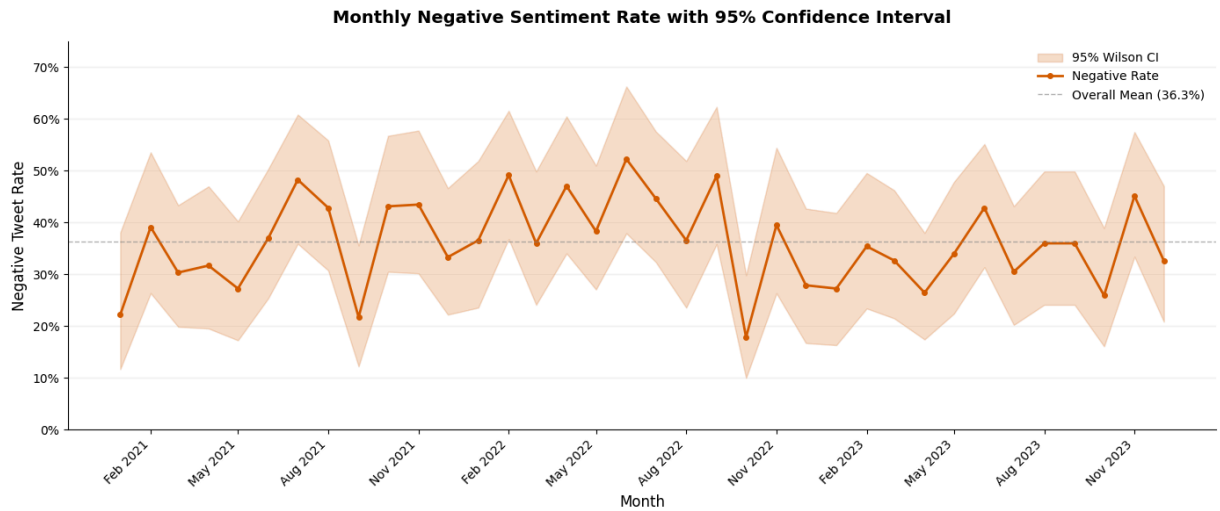
```
      month  n_negative  n_total      rate    ci_low   ci_high
0   2021-01           8       36  0.222222  0.117162  0.380850
1   2021-02          18       46  0.391304  0.263930  0.535434
2   2021-03          17       56  0.303571  0.199002  0.433361
3   2021-04          13       41  0.317073  0.195644  0.469845
4   2021-05          15       55  0.272727  0.172806  0.402324
5   2021-06          20       54  0.370370  0.254232  0.503728
6   2021-07          28       58  0.482759  0.359280  0.608380
7   2021-08          24       56  0.428571  0.307688  0.558626
8   2021-09          10       46  0.217391  0.122608  0.355740
9   2021-10          22       51  0.431373  0.305010  0.567349
10  2021-11          20       46  0.434783  0.302090  0.577528
11  2021-12          18       54  0.333333  0.222412  0.466393
```

Method: I use the Wilson score interval rather than the normal approximation because it provides better coverage for proportions near 0 or 1 and for months with smaller sample sizes. The Wilson interval accounts for the bounded nature of proportions and never produces impossible values below 0 or above 1. At z = 1.96 (95% confidence), each monthly point has a CI that reflects both the estimated proportion and the sample size for that month.

In [25]:

```
# time-series: monthly negative sentiment rate with 95% CI
fig, ax = plt.subplots(figsize=(14, 6))

# confidence band
ax.fill_between(monthly['month_dt'], monthly['ci_low'], monthly['ci_high'],
                alpha=0.2, color='#D55E00', label='95% Wilson CI')

# rate line
ax.plot(monthly['month_dt'], monthly['rate'],
        color='#D55E00', linewidth=2, marker='o', markersize=4, label='Negat

# overall mean reference line
overall_rate = df['is_negative'].mean()
ax.axhline(y=overall_rate, color='gray', linestyle='--', linewidth=1, alpha=
           label=f'Overall Mean ({overall_rate:.1%})')

ax.set_xlabel('Month', fontsize=12)
ax.set_ylabel('Negative Tweet Rate', fontsize=12)
ax.set_title('Monthly Negative Sentiment Rate with 95% Confidence Interval',
             fontsize=14, fontweight='bold', pad=15)
ax.legend(frameon=False, fontsize=10, loc='upper right')
ax.tick_params(axis='both', labelsize=10)

# format axes
ax.xaxis.set_major_locator(plt.matplotlib.dates.MonthLocator(interval=3))
ax.xaxis.set_major_formatter(plt.matplotlib.dates.DateFormatter('%b %Y'))
plt.setp(ax.xaxis.get_majorticklabels(), rotation=45, ha='right')

ax.yaxis.set_major_formatter(plt.matplotlib.ticker.PercentFormatter(1.0))

for spine in ['top', 'right']:
    ax.spines[spine].set_visible(False)
```

```
ax.yaxis.grid(True, linestyle='-', alpha=0.15)
ax.set_axisbelow(True)
ax.set_ylim(0, 0.75)

plt.tight_layout()
plt.show()
```



The plot shows that negative sentiment fluctuates between roughly 18% and 52% month to month. The CI bands are wider in months with fewer tweets and narrower in high-volume months, which is expected. There is no obvious long-term upward or downward trend in negative sentiment over the 2021 to 2023 window, though individual months show notable spikes (e.g. July 2021 at ~48%, June 2022 at ~52%).

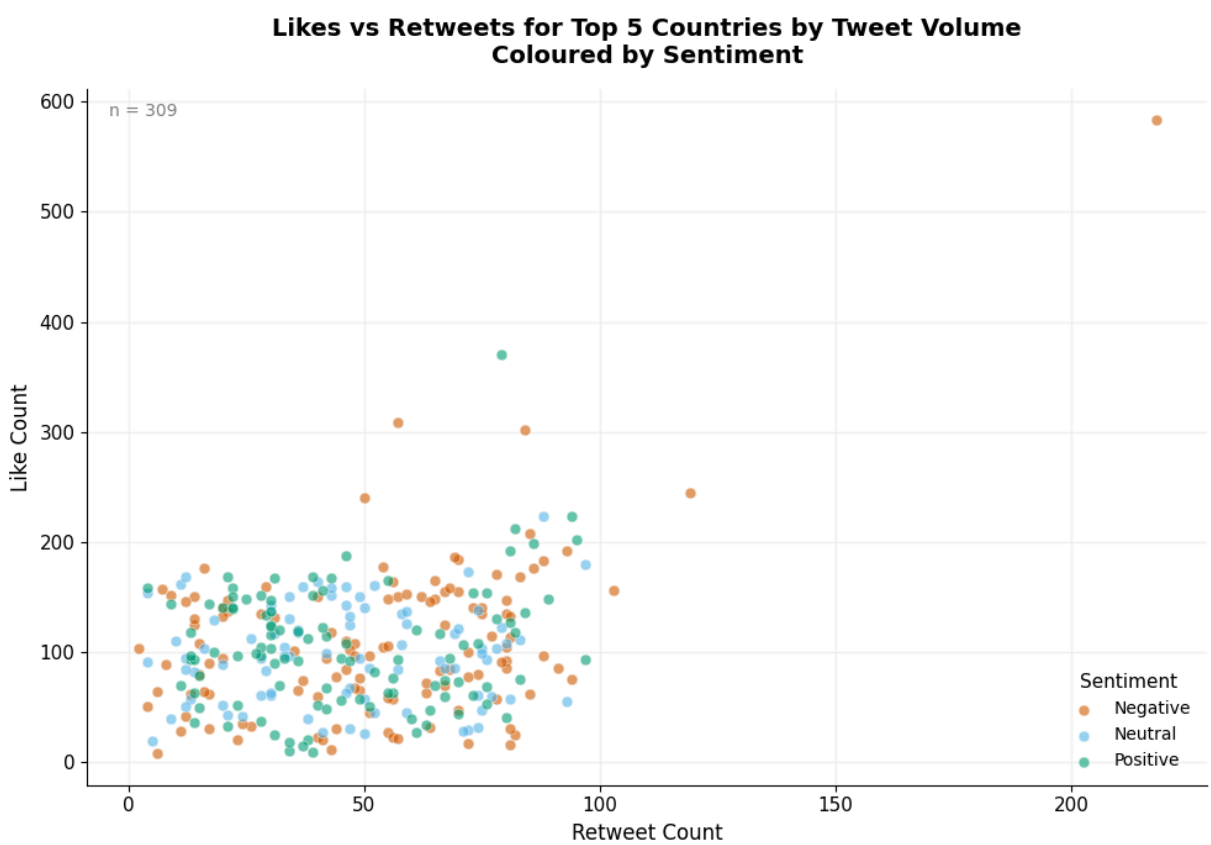
D2 - Scatter/Bubble: Likes vs Retweets

Select the top 5 countries by tweet volume to avoid an overly dense plot.

In [26]:
```
# select top 5 countries by tweet count
top5 = df['country'].value_counts().head(5).index.tolist()
df_scatter = df[df['country'].isin(top5)].copy()

print(f"Top 5 countries: {top5}")
print(f"Rows for scatter: {len(df_scatter)}")
print(f"\nSentiment breakdown in subset:")
print(df_scatter['sentiment'].value_counts())
```

```
Top 5 countries: ['New Zealand', 'Thailand', 'Mexico', 'Spain', 'Denmark']
Rows for scatter: 309

Sentiment breakdown in subset:
sentiment
negative    119
positive    104
neutral      86
Name: count, dtype: int64
```

In [27]:
```
# scatter: likes vs retweets, coloured by sentiment
fig, ax = plt.subplots(figsize=(10, 7))

for sentiment, color in sentiment_colors.items():
    subset = df_scatter[df_scatter['sentiment'] == sentiment]
    ax.scatter(subset['retweet_count'], subset['like_count'],
               c=color, label=sentiment.capitalize(), alpha=0.6, s=40,
               edgecolors='white', linewidths=0.5)

ax.set_xlabel('Retweet Count', fontsize=12)
ax.set_ylabel('Like Count', fontsize=12)
ax.set_title('Likes vs Retweets for Top 5 Countries by Tweet Volume\nColoure
             fontsize=14, fontweight='bold', pad=15)
ax.legend(title='Sentiment', frameon=False, fontsize=10, title_fontsize=11)
ax.tick_params(axis='both', labelsize=11)

for spine in ['top', 'right']:
    ax.spines[spine].set_visible(False)

ax.xaxis.grid(True, linestyle='-', alpha=0.15)
ax.yaxis.grid(True, linestyle='-', alpha=0.15)
ax.set_axisbelow(True)

# sample size
ax.text(0.02, 0.98, f'n = {len(df_scatter)}', transform=ax.transAxes,
        fontsize=10, va='top', color='gray')

plt.tight_layout()
plt.show()
```



Discussion: The scatter plot shows a broadly positive association between retweet count and like count, which is expected since both measure engagement. Sentiment doesn't cluster in any particular region of the engagement space: negative, neutral, and positive tweets are all scattered across the full range of likes and retweets. This suggests that engagement levels are driven more by topic or timing than by whether the tweet is framed positively or negatively. There is no evidence that negative tweets attract disproportionately more engagement than positive ones in this subset.

Part E - Design & Insight

E1 - Design Choices

Colour Scales All visualizations use the Okabe-Ito palette, designed for colourblind accessibility (I am colourblind). Sentiment is consistently encoded across all charts: red-orange (#D55E00) for negative, sky blue (#56B4E9) for neutral, and green (#009E73) for positive. The choropleth uses a sequential Blues scale where darker shading corresponds to higher tweet counts, creating an intuitive "more = darker" mapping. Word clouds use green (positive) and orange-red (negative) colourmaps to reinforce the sentiment association.

Uncertainty The Wilson score interval was chosen over the normal approximation for the monthly negative rate because it handles small-sample months better and never produces CI bounds outside [0, 1]. The shaded band encodes uncertainty continuously rather than using error bars, reducing visual clutter and making temporal trends easier to follow.

Cognitive Load Horizontal bars eliminate rotated text labels. Chart junk (borders, unnecessary gridlines, heavy spines) is removed throughout. Value labels are placed directly beside or above bars to reduce eye movement. The scatter plot uses transparency (alpha=0.6) and white edge markers to manage overplotting in dense regions.

Contiguity Labels are placed adjacent to their data points across all charts. The CI band sits directly on the rate line. Legend placement is consistent and near the relevant data region.

E2 - Key Insights

Negative sentiment is the most prevalent category (36.3%), slightly outpacing positive (33.0%) and neutral (30.7%), suggesting climate discourse on social media leans pessimistic overall. This distribution is not uniform across countries:

Mexico and Saudi Arabia show substantially higher negative proportions, while Italy has the strongest positive skew among the top 10 countries. The word frequency analysis confirms that the positive-negative split is driven by distinct framing language: positive tweets cluster around "encouraging", "progress", and "boost", while negative tweets emphasize "concerns", "worsening", and "setbacks", even when discussing the same underlying topics. The monthly CI plot shows no clear temporal trend in negative sentiment over the three-year window, which suggests that the overall pessimistic tilt is structural rather than driven by specific events. Finally, the engagement analysis reveals that sentiment does not predict virality: likes and retweets are distributed similarly across all three sentiment categories, implying that content topic and timing matter more than emotional framing for driving engagement.